

Übungseinheit: Programmverzweigungen und Schleifen

Entscheidungen treffen und Wiederholungen automatisieren

Dorian Zwanzig, Claude

19. November 2025

Inhaltsverzeichnis

1 Über diese Übung	3
2 Vorbereitung	4
3 Wiederholung Programmverzweigungen (Notebook 09)	5
4 Block A – if und if-else (Einfach)	6
4.1 Aufgabe 1: Produktionsüberwachung	6
4.2 Aufgabe 2: Temperaturüberwachung	7
4.3 Aufgabe 3: Bauteil-ID Klassifizierung	7
5 Block B – elif für mehrere Fälle (Mittel)	9
5.1 Aufgabe 4: Qualitätskontrolle	9
5.2 Aufgabe 5: Maschinendrehzahl klassifizieren	10
5.3 Aufgabe 6: Lagerverwaltung – Füllstand prüfen	11
6 Block C – match-case für Fallunterscheidung (Mittel)	13
6.1 Aufgabe 7: Maschinentyp-Auswahl	13
6.2 Aufgabe 8: Schichtplanung	14
7 Wiederholung Schleifen (Notebook 10)	16
8 Block D – while-Schleifen (Einfach-Mittel)	17
8.1 Aufgabe 9: Countdown für Maschinenstart	17
8.2 Aufgabe 10: Eingabe-Validierung für Temperatur	18
8.3 Aufgabe 11: Produktionsdaten sammeln	19
9 Block E – for-Schleifen (Einfach-Mittel)	21
9.1 Aufgabe 12: Maximum aus Messwerten finden	21
9.2 Aufgabe 13: Defekte Bauteile zählen	22
9.3 Aufgabe 14: Lagerbericht erstellen	22
10 Block F – break und continue (Mittel)	25
10.1 Aufgabe 15: Erste defekte Charge finden	25
10.2 Aufgabe 16: Nur gültige Messwerte verarbeiten	26
11 Komplexaufgaben (Block G und H)	28
12 Block G – Komplexaufgabe 1	28
12.1 Aufgabe 17: Produktionslinien-Analyse	28
13 Block H – Komplexaufgabe 2	29
13.1 Aufgabe 18: Qualitätsbericht-Generator	29

14 Musterlösungen	32
15 Typische Anfängerfehler – Troubleshooting	50
16 Abschluss & Reflexion	51

1 Über diese Übung

Lernziele

Nach Abschluss dieser Übung können Sie:

- **if, if-else und elif-Anweisungen** schreiben und anwenden
- **match-case-Strukturen** für Fallunterscheidungen nutzen
- **while-Schleifen** für bedingte Wiederholungen implementieren
- **for-Schleifen** zur Iteration über Kollektionen verwenden
- **break und continue** zur Schleifensteuerung einsetzen
- **Verzweigungen und Schleifen kombinieren** für komplexe Problemlösungen
- **Praktische Anwendungen** im ingenieurwissenschaftlichen Kontext umsetzen

Zeitbedarf: ca. 120-180 Minuten **Schwierigkeitsgrad:** (Einfach bis Mittel, mit Herausforderungen)

Voraussetzungen

Für diese Übung sollten Sie folgende Konzepte sicher beherrschen:

- **Variablen und einfache Datentypen** (Notebooks 04-05)
- **Listen und Dictionaries** (Notebook 06)
- **Funktionen** (Notebook 07)
- **Operatoren** (Notebook 08: Vergleichs-, logische und arithmetische Operatoren)

Diese Übung vertieft die Konzepte aus:

- **Notebook 09: Programmverzweigungen**
- **Notebook 10: Schleifen**

Falls Sie unsicher sind, wiederholen Sie bitte die entsprechenden Notebooks.

Hinweis zur Bearbeitung

Dateiorganisation:

Legen Sie für jeden Übungsblock eine eigene Python-Datei an:

- `block_a_if_else.py`
- `block_b_elif.py`
- `block_c_match_case.py`
- `block_d_while.py`
- `block_e_for.py`
- `block_f_break_continue.py`
- `komplex_aufgaben.py`

Das hilft Ihnen, die Übersicht zu behalten – heute und bei der Klausurvorbereitung.

2 Vorbereitung

Aufwand: ca. 10 Minuten

Technische Vorbereitung

1. Erstellen Sie einen neuen Ordner `uebung_verzweigungen_schleifen`
2. Öffnen Sie Ihre Python-Entwicklungsumgebung (VS Code, PyCharm, etc.)
3. Erstellen Sie die erste Übungsdatei `umgebungstest.py`

Mentale Vorbereitung

In dieser Übung werden Sie:

1. **Entscheidungen treffen:** Programme schreiben, die auf Bedingungen reagieren
2. **Wiederholungen automatisieren:** Aufgaben mit Schleifen effizient lösen
3. **Beide Konzepte kombinieren:** Komplexe Problemstellungen strukturiert angehen

Wichtig: Konzentrieren Sie sich auf **klare, lesbare Lösungen**. Clevere Tricks sind weniger wichtig als verständlicher Code!

Test der Umgebung

Führen Sie folgenden Code in `umgebungstest.py` aus:

```
# Umgebungstest
print("Python-Umgebung bereit!")

# Test: if-else
zahl = 10
if zahl > 5:
    print("[OK] if-else funktioniert")
else:
    print("[FEHLER] if-else Fehler")

# Test: while-Schleife
counter = 0
while counter < 3:
    counter += 1
print(f"[OK] while-Schleife durchlief {counter} Mal")

# Test: for-Schleife
zahlen = [1, 2, 3]
summe = 0
for z in zahlen:
    summe += z
print(f"[OK] for-Schleife berechnete Summe: {summe}")

print("\n[OK] Alle Tests bestanden - Sie können beginnen!")
```

Erwartete Ausgabe:

```
Python-Umgebung bereit!
[OK] if-else funktioniert
[OK] while-Schleife durchlief 3 Mal
[OK] for-Schleife berechnete Summe: 6
[OK] Alle Tests bestanden - Sie können beginnen!
```

3 Wiederholung Programmverzweigungen (Notebook 09)

if-Anweisung: Code nur unter bestimmten Bedingungen ausführen

```
if bedingung:  
    # wird ausgeführt, wenn bedingung True ist
```

if-else-Anweisung: Zwischen zwei Alternativen entscheiden

```
if bedingung:  
    # wenn True  
else:  
    # wenn False
```

elif-Anweisung: Mehrere Bedingungen nacheinander prüfen

```
if bedingung1:  
    # Fall 1  
elif bedingung2:  
    # Fall 2  
else:  
    # Alle anderen Fälle
```

match-case-Anweisung: Elegante Fallunterscheidung (Python 3.10+)

```
match variable:  
    case wert1:  
        # Für wert1  
    case wert2:  
        # Für wert2  
    case _:  
        # Standard-Fall
```

4 Block A – if und if-else (Einfach)

Aufwand: ca. 15-20 Minuten

Erstellen Sie die Datei `block_a_if_else.py` für die folgenden Aufgaben.

Info-Box: Vergleichsoperatoren

Sie benötigen folgende Operatoren: - > größer als - < kleiner als - >= größer oder gleich - <= kleiner oder gleich - == gleich - != ungleich

4.1 Aufgabe 1: Produktionsüberwachung

Schwierigkeit: (Einfach)

Aufgabenstellung

In einer Produktionshalle werden täglich Bauteile gefertigt. Das Tagesziel liegt bei 100 Stück.

Schreiben Sie ein Programm, das:

1. Die tatsächlich produzierte Stückzahl vom Benutzer abfragt
2. Prüft, ob das Tagesziel erreicht wurde
3. Eine entsprechende Meldung ausgibt:
 - "Tagesziel erreicht!" (wenn ≥ 100)
 - "Tagesziel nicht erreicht." (wenn < 100)

Erwartetes Verhalten:

Produzierte Stückzahl: 120

Tagesziel erreicht!

Hilfestellungen

Problemanalyse:

- Welche Entscheidung muss getroffen werden? -> Ziel erreicht oder nicht
- Wie viele Fälle gibt es? -> Genau zwei (erreicht / nicht erreicht)
- Welche Kontrollstruktur passt? -> if-else

Lösungsschritte:

1. Eingabe vom Benutzer holen (mit `input()` und Konvertierung zu `int`)
2. Variable mit Tagesziel definieren (100)
3. Bedingung formulieren: Ist produzierte Stückzahl $\geq$ Tagesziel?
4. if-else-Struktur aufbauen
5. Entsprechende Nachrichten ausgeben

Pseudocode:

```
tagesziel = 100
produziert = EINGABE VON BENUTZER (als Ganzzahl)
```

```
WENN produziert >= tagesziel
    DANN gebe "Tagesziel erreicht!" aus
SONST
    DANN gebe "Tagesziel nicht erreicht." aus
```

4.2 Aufgabe 2: Temperaturüberwachung

Schwierigkeit: (Einfach)

Aufgabenstellung

Eine CNC-Maschine darf eine Betriebstemperatur von 80°C nicht überschreiten. Entwickeln Sie ein Überwachungsprogramm.

Das Programm soll:

1. Die aktuelle Temperatur vom Benutzer abfragen
2. Eine Warnung ausgeben, wenn die Temperatur > 80°C ist
3. "Temperatur im Normbereich" ausgeben, wenn die Temperatur <= 80°C ist

Erwartetes Verhalten:

Aktuelle Temperatur (°C): 85

[WARNUNG] WARNUNG: Überhitzung! Maschine prüfen!

Aktuelle Temperatur (°C): 75

Temperatur im Normbereich.

Hilfestellungen

Problemanalyse:

- Schwellenwert: 80°C
- Zwei Fälle: Überschritten oder nicht
- Struktur: if-else

Lösungsschritte:

1. Grenzwert als Variable definieren (80)
2. Temperatur vom Benutzer abfragen (als float)
3. Vergleich: Ist Temperatur > Grenzwert?
4. Entsprechende Meldung ausgeben

Pseudocode:

```
grenzwert = 80
temperatur = EINGABE VON BENUTZER (als Dezimalzahl)
```

```
WENN temperatur > grenzwert
    DANN gebe Warnung aus
SONST
    DANN gebe "Normbereich" aus
```

4.3 Aufgabe 3: Bauteil-ID Klassifizierung

Schwierigkeit: (Einfach)

Aufgabenstellung

In einem Lager werden Bauteile mit IDs versehen. Gerade IDs sind für Lager A, ungerade IDs für Lager B bestimmt.

Schreiben Sie ein Programm, das: 1. Eine Bauteil-ID vom Benutzer abfragt 2. Prüft, ob die ID gerade oder ungerade ist 3. Ausgibt, in welches Lager das Bauteil gehört

Hinweis: Eine Zahl ist gerade, wenn `zahl % 2 == 0`

Erwartetes Verhalten:

Bauteil-ID: 1042

Lager A (gerade ID)

Bauteil-ID: 1043

Lager B (ungerade ID)

Hilfestellungen**Problemanalyse:**

- Gerade oder ungerade? -> Modulo-Operator %
- Zwei Lager, zwei Fälle -> if-else

Lösungsschritte:

1. Bauteil-ID vom Benutzer abfragen (als int)
2. Modulo 2 berechnen: $id \% 2$
3. Wenn Ergebnis 0 -> gerade -> Lager A
4. Sonst -> ungerade -> Lager B

Pseudocode:

bauteil_id = EINGABE VON BENUTZER (als Ganzzahl)

WENN bauteil_id % 2 == 0

 DANN gebe "Lager A (gerade ID)" aus

SONST

 DANN gebe "Lager B (ungerade ID)" aus

Konzept-Hinweis:

Der Modulo-Operator % gibt den Rest einer Division zurück: - $4 \% 2 = 0$ (gerade, kein Rest) - $5 \% 2 = 1$ (ungerade, Rest 1)

5 Block B – elif für mehrere Fälle (Mittel)

Aufwand: ca. 20-30 Minuten

Erstellen Sie die Datei `block_b_elif.py` für die folgenden Aufgaben.

Info-Box: elif-Struktur

Verwenden Sie `elif`, wenn Sie **mehr als zwei Fälle** unterscheiden müssen:

```
if bedingung1:
    # Fall 1
elif bedingung2:
    # Fall 2
elif bedingung3:
    # Fall 3
else:
    # Alle anderen Fälle
```

Python prüft die Bedingungen **von oben nach unten** und führt nur den **ersten zutreffenden** Block aus.

5.1 Aufgabe 4: Qualitätskontrolle

Schwierigkeit: (Mittel)

Aufgabenstellung

In einer Produktionslinie wird die Fehlerquote überwacht. Entwickeln Sie ein Bewertungssystem.

Das Programm soll:

1. Die Anzahl produzierter Teile abfragen
2. Die Anzahl fehlerhafter Teile abfragen
3. Die Fehlerquote in Prozent berechnen: $(\text{fehlerhafte} / \text{produzierte}) * 100$
4. Eine Bewertung ausgeben:
 - **< 1%:** "Exzellente Qualität"
 - **1% - 2.9%:** "Gute Qualität"
 - **3% - 4.9%:** "Akzeptable Qualität – Optimierung empfohlen"
 - **>= 5%:** "Kritisch – Sofortige Maßnahmen erforderlich!"

Erwartetes Verhalten:

```
Produzierte Teile: 1000
Fehlerhafte Teile: 15
Fehlerquote: 1.5%
Bewertung: Gute Qualität
```

Hilfestellungen

Problemanalyse:

- Vier verschiedene Qualitätsstufen -> vier Fälle
- Bedingungen sind Bereiche (< 1, 1-3, 3-5, >= 5) -> `elif`
- Reihenfolge wichtig: Von klein nach groß prüfen

Lösungsschritte:

1. Beide Eingaben abfragen (produzierte und fehlerhafte Teile)
2. Fehlerquote berechnen: $(\text{fehlerhafte} / \text{produzierte}) * 100$
3. Vier Bedingungen mit `if-elif-elif-else` prüfen

4. Fehlerquote und Bewertung ausgeben

Pseudocode:

```
produzierte = EINGABE (Ganzzahl)
fehlerhafte = EINGABE (Ganzzahl)
fehlerquote = (fehlerhafte / produzierte) * 100
```

Gebe fehlerquote aus

```
WENN fehlerquote < 1
    DANN gebe "Exzellente Qualität" aus
SONST WENN fehlerquote < 3
    DANN gebe "Gute Qualität" aus
SONST WENN fehlerquote < 5
    DANN gebe "Akzeptable Qualität" aus
SONST
    DANN gebe "Kritisch" aus
```

Hinweis zur Reihenfolge:

Prüfen Sie immer von der kleinsten Bedingung zur größten:

- Wenn Sie zuerst < 5 prüfen, wäre < 1 nie erreichbar (weil < 1 auch < 5 ist)!
-

5.2 Aufgabe 5: Maschinendrehzahl klassifizieren

Schwierigkeit: (Mittel)

Aufgabenstellung

Eine Drehmaschine hat verschiedene Betriebsmodi abhängig von der Drehzahl (U/min - Umdrehungen pro Minute).

Schreiben Sie ein Programm, das:

1. Die aktuelle Drehzahl abfragt
2. Den Betriebsmodus bestimmt:
 - **< 500 U/min:** "Standby-Modus"
 - **500 - 1499 U/min:** "Niedrige Drehzahl"
 - **1500 - 2999 U/min:** "Normal-Betrieb"
 - **>= 3000 U/min:** "Hochleistungsmodus"

Erwartetes Verhalten:

Aktuelle Drehzahl (U/min): 2100
Betriebsmodus: Normal-Betrieb

Hilfestellungen**Problemanalyse:**

- Vier Bereiche -> vier Fälle
- Grenzen: 500, 1500, 3000
- Struktur: if-elif-elif-else

Lösungsschritte:

1. Drehzahl abfragen
2. Von klein nach groß prüfen:
 - Erst < 500

- Dann < 1500 (wenn >= 500)
- Dann < 3000 (wenn >= 1500)
- Sonst >= 3000

3. Entsprechenden Modus ausgeben

Pseudocode:

drehzahl = EINGABE (Ganzzahl)

```

WENN drehzahl < 500
    DANN "Standby-Modus"
SONST WENN drehzahl < 1500
    DANN "Niedrige Drehzahl"
SONST WENN drehzahl < 3000
    DANN "Normal-Betrieb"
SONST
    DANN "Hochleistungsmodus"

```

5.3 Aufgabe 6: Lagerverwaltung – Füllstand prüfen

Schwierigkeit: (Mittel)

Aufgabenstellung

Ein Lager für Rohstoffe hat eine Kapazität von 1000 Einheiten. Der Füllstand soll überwacht werden.

Das Programm soll:

1. Den aktuellen Bestand abfragen
2. Den Füllstand in Prozent berechnen: $(\text{bestand} / 1000) * 100$
3. Eine Statusmeldung ausgeben:
 - < 25%: "Kritisch niedrig – Nachbestellung erforderlich!"
 - 25% - 49%: "Niedrig – Bald nachbestellen"
 - 50% - 89%: "Normaler Füllstand"
 - >= 90%: "Fast voll – Platzmangel prüfen"

Erwartetes Verhalten:

Aktueller Bestand: 320
 Füllstand: 32.0%
 Status: Niedrig – Bald nachbestellen

Hilfestellungen

Problemanalyse:

- Vier Füllstandsbereiche
- Berechnung notwendig: Prozentsatz aus absolutem Wert
- elif-Kette für Bereiche

Lösungsschritte:

1. Kapazität als Konstante definieren (1000)
2. Aktuellen Bestand abfragen
3. Füllstand in Prozent berechnen
4. Mit if-elif-elif-else vier Bereiche abfragen
5. Füllstand und Status ausgeben

Pseudocode:

```
kapazitaet = 1000
bestand = EINGABE (Ganzzahl)
fuellstand = (bestand / kapazitaet) * 100
```

Gebe fuellstand aus

```
WENN fuellstand < 25
    DANN "Kritisch niedrig"
SONST WENN fuellstand < 50
    DANN "Niedrig"
SONST WENN fuellstand < 90
    DANN "Normaler Füllstand"
SONST
    DANN "Fast voll"
```

6 Block C – match-case für Fallunterscheidung (Mittel)

Aufwand: ca. 15-20 Minuten

Erstellen Sie die Datei `block_c_match_case.py` für die folgenden Aufgaben.

Info-Box: match-case

match-case ist ideal, wenn Sie eine Variable mit **vielen spezifischen Werten** vergleichen:

```
match variable:
    case "wert1":
        # Code für wert1
    case "wert2" | "wert3": # Mehrere Werte mit |
        # Code für wert2 oder wert3
    case _: # Unterstrich = alle anderen Fälle
        # Standard-Fall
```

Wichtig: match-case benötigt Python 3.10 oder höher!

6.1 Aufgabe 7: Maschinentyp-Auswahl

Schwierigkeit: (Mittel)

Aufgabenstellung

In einer Produktionshalle stehen verschiedene Maschinentypen zur Verfügung. Jeder Typ hat spezifische Eigenschaften.

Schreiben Sie ein Programm, das:

1. Den Maschinentyp vom Benutzer abfragt (Buchstabe A-E)
2. Mit match-case die Eigenschaften ausgibt:
 - **A:** "CNC-Fräse – Maximale Genauigkeit: 0.01mm"
 - **B:** "Drehmaschine – Maximale Drehzahl: 3000 U/min"
 - **C:** "Laserschneider – Maximale Leistung: 2000W"
 - **D:** "3D-Drucker – Druckvolumen: 300x300x400mm"
 - **E:** "Schweißroboter – Reichweite: 2.5m"
 - **Anderer Wert:** "Unbekannter Maschinentyp"

Erwartetes Verhalten:

```
Maschinentyp (A-E): C
Laserschneider – Maximale Leistung: 2000W
```

Hilfestellungen

Problemanalyse:

- Fünf konkrete Werte (A, B, C, D, E) + Standard-Fall
- Perfekt für match-case (keine Bereiche, nur exakte Werte)

Lösungsschritte:

1. Maschinentyp als String eingeben lassen
2. Optional: Mit `.upper()` in Großbuchstaben umwandeln (dann funktioniert auch "a")
3. match-case mit fünf cases und einem Standard-Fall
4. Für jeden case die entsprechende Info ausgeben

Pseudocode:

```
typ = EINGABE (als String, in Großbuchstaben)
```

```
VERGLEICHE typ MIT
```

```
  FALL "A":
    Gebe "CNC-Fräse..." aus
  FALL "B":
    Gebe "Drehmaschine..." aus
  FALL "C":
    Gebe "Laserschneider..." aus
  FALL "D":
    Gebe "3D-Drucker..." aus
  FALL "E":
    Gebe "Schweißroboter..." aus
  STANDARD-FALL:
    Gebe "Unbekannt" aus
```

6.2 Aufgabe 8: Schichtplanung

Schwierigkeit: (Mittel)

Aufgabenstellung

Ein Produktionsbetrieb arbeitet in drei Schichten. Jede Schicht hat feste Arbeitszeiten und einen Zuschlag.

Schreiben Sie ein Programm, das: 1. Die Schichtnummer (1, 2 oder 3) abfragt 2. Mit match-case folgende Informationen ausgibt: - **Schicht 1:** "Frühschicht: 06:00 - 14:00 Uhr, Zuschlag: 0%" - **Schicht 2:** "Spätschicht: 14:00 - 22:00 Uhr, Zuschlag: 10%" - **Schicht 3:** "Nachtschicht: 22:00 - 06:00 Uhr, Zuschlag: 25%" - **Anderer Wert:** "Ungültige Schichtnummer"

Erwartetes Verhalten:

```
Schichtnummer (1-3): 2
Spätschicht: 14:00 - 22:00 Uhr, Zuschlag: 10%
```

Hilfestellungen

Problemanalyse:

- Drei konkrete Werte (1, 2, 3)
- match-case mit Zahlen statt Strings

Lösungsschritte:

1. Schichtnummer als int eingeben lassen
2. match-case mit drei Fällen + Standard
3. Jeweils Zeiten und Zuschlag ausgeben

Pseudocode:

```
schicht = EINGABE (als Ganzzahl)
```

```
VERGLEICHE schicht MIT
```

```
  FALL 1:
    Gebe "Frühschicht..." aus
  FALL 2:
    Gebe "Spätschicht..." aus
  FALL 3:
    Gebe "Nachtschicht..." aus
```

STANDARD:

Gebe "Ungültig" aus

Hinweis:

match-case funktioniert mit allen Datentypen: Strings, Zahlen, sogar mit Tupeln!

7 Wiederholung Schleifen (Notebook 10)

while-Schleife: Wiederholt, solange Bedingung erfüllt ist

```
while bedingung:  
    # Wird wiederholt, solange bedingung True ist
```

for-Schleife: Iteriert über alle Elemente einer Kollektion

```
for element in kollektion:  
    # Wird für jedes Element ausgeführt
```

Schleifensteuerung: - break -> Beendet Schleife sofort - continue -> Überspringt aktuellen Durchlauf

8 Block D – while-Schleifen (Einfach-Mittel)

Aufwand: ca. 20-30 Minuten

Erstellen Sie die Datei `block_d_while.py` für die folgenden Aufgaben.

Info-Box: while-Schleifen

Eine while-Schleife wiederholt Code, **solange** eine Bedingung erfüllt ist:

```
while bedingung:
    # Code wird wiederholt
    # WICHTIG: Bedingung muss irgendwann False werden!
```

Achtung vor Endlosschleifen! Die Variable in der Bedingung muss sich ändern.

8.1 Aufgabe 9: Countdown für Maschinenstart

Schwierigkeit: (Einfach)

Aufgabenstellung

Eine Produktionsmaschine benötigt einen Countdown vor dem Start.

Schreiben Sie ein Programm, das:

1. Von 10 bis 1 herunterzählt
2. Jede Sekunde (Zahl) ausgibt
3. Nach dem Countdown "START!" ausgibt

Erwartetes Verhalten:

```
10
9
8
7
6
5
4
3
2
1
START!
```

Hilfestellungen

Problemanalyse:

- Wiederholung mit bekannter Anzahl (10 Mal)
- Zähler wird verringert
- While-Schleife mit Bedingung "solange > 0"

Lösungsschritte:

1. Variable `countdown` mit Wert 10 initialisieren
2. while-Schleife mit Bedingung `countdown >= 1` (oder `> 0`)
3. In der Schleife: Zahl ausgeben
4. Countdown um 1 verringern
5. Nach der Schleife: "START!" ausgeben

Pseudocode:

```
countdown = 10

SOLANGE countdown >= 1
    Gebe countdown aus
    countdown = countdown - 1
```

Gebe "START!" aus

Wichtiger Hinweis:

Vergessen Sie nicht, `countdown` zu verringern! Sonst läuft die Schleife ewig.

8.2 Aufgabe 10: Eingabe-Validierung für Temperatur

Schwierigkeit: (Mittel)

Aufgabenstellung

Ein Steuerungssystem akzeptiert nur Temperaturen zwischen 0°C und 200°C. Ungültige Eingaben sollen wiederholt abgefragt werden.

Schreiben Sie ein Programm, das:

1. So lange nach einer Temperatur fragt, bis eine gültige Eingabe gemacht wird
2. Bei ungültiger Eingabe eine Fehlermeldung ausgibt
3. Bei gültiger Eingabe die Temperatur bestätigt und die Schleife beendet

Erwartetes Verhalten:

```
Temperatur (0-200°C): 250
Ungültige Eingabe! Temperatur muss zwischen 0 und 200°C liegen.
Temperatur (0-200°C): -10
Ungültige Eingabe! Temperatur muss zwischen 0 und 200°C liegen.
Temperatur (0-200°C): 150
Temperatur akzeptiert: 150.0°C
```

Hilfestellungen**Problemanalyse:**

- Wiederholung, bis gültige Eingabe
- Anzahl der Wiederholungen unbekannt -> while-Schleife
- Zwei Möglichkeiten: `while True` mit `break` ODER `while` mit Bedingung

Lösungsschritte (Variante 1 – mit while True):

1. Endlosschleife mit `while True`: starten
2. Temperatur abfragen
3. Prüfen: Ist sie zwischen 0 und 200?
4. Wenn ja: Bestätigung ausgeben und `break`
5. Wenn nein: Fehlermeldung ausgeben, Schleife läuft weiter

Pseudocode:

```
ENDLOSSCHLEIFE
    temp = EINGABE (Dezimalzahl)

    WENN temp >= 0 UND temp <= 200
        Gebe "Akzeptiert" aus
    BEENDE SCHLEIFE (break)
```

```
SONST
    Gebe Fehlermeldung aus
```

Alternative (Variante 2 – mit Bedingung):

```
gueltig = False
```

```
SOLANGE NICHT gueltig
    temp = EINGABE
```

```
WENN temp zwischen 0 und 200
    gueltig = True
    Gebe "Akzeptiert" aus
SONST
    Gebe Fehlermeldung aus
```

Tipp: Variante 1 ist kürzer und eleganter!

8.3 Aufgabe 11: Produktionsdaten sammeln

Schwierigkeit: (Mittel)

Aufgabenstellung

Ein Produktionsmitarbeiter soll Stückzahlen für verschiedene Produkte eingeben, bis "STOP" eingegeben wird.

Schreiben Sie ein Programm, das: 1. Wiederholt nach einem Produktnamen fragt 2. Bei "STOP" (Groß-/Kleinschreibung egal) die Eingabe beendet 3. Sonst nach der Stückzahl fragt 4. Am Ende die Gesamtzahl aller Stücke ausgibt

Erwartetes Verhalten:

```
Produktname (oder STOP): Schrauben
Stückzahl: 150
Produktname (oder STOP): Muttern
Stückzahl: 200
Produktname (oder STOP): STOP
```

```
Gesamtzahl produzierte Teile: 350
```

Hilfestellungen

Problemanalyse:

- Wiederholung bis Abbruchbedingung ("STOP")
- Zähler für Gesamtstückzahl notwendig
- while-Schleife mit Bedingung oder while True mit break

Lösungsschritte:

1. Variable `gesamtstueckzahl` = 0 initialisieren
2. Endlosschleife starten
3. Produktnamen abfragen
4. Prüfen: Ist es "STOP" (oder "stop", "Stop")? -> Mit `.upper()` oder `.lower()`
5. Wenn STOP: Schleife beenden
6. Sonst: Stückzahl abfragen und zur Gesamtsumme addieren
7. Nach Schleife: Gesamtsumme ausgeben

Pseudocode:

```
gesamt = 0
```

```
ENDLOSSCHLEIFE
```

```
    produkt = EINGABE (String)
```

```
    WENN produkt in Großbuchstaben == "STOP"
```

```
        BEENDE SCHLEIFE
```

```
    stueckzahl = EINGABE (Ganzzahl)
```

```
    gesamt = gesamt + stueckzahl
```

```
Gebe gesamt aus
```

Hinweis zu .upper():

"stop".upper() ergibt "STOP" – so funktioniert die Prüfung unabhängig von Groß-/Kleinschreibung!

9 Block E – for-Schleifen (Einfach-Mittel)

Aufwand: ca. 20-30 Minuten

Erstellen Sie die Datei `block_e_for.py` für die folgenden Aufgaben.

Info-Box: for-Schleifen

Eine for-Schleife iteriert über **alle Elemente** einer Kollektion:

```
for element in liste:  
    # Code wird für jedes Element ausgeführt
```

Funktioniert mit: Listen, Strings, Dictionaries (`.items()`), `range()`

9.1 Aufgabe 12: Maximum aus Messwerten finden

Schwierigkeit: (Einfach)

Aufgabenstellung

Ein Sensor hat 5 Temperaturmesswerte aufgezeichnet. Finden Sie den höchsten Wert **ohne** die `max()`-Funktion zu verwenden.

Gegeben ist:

```
messwerte = [23.5, 25.1, 22.8, 26.3, 24.0]
```

Das Programm soll: 1. Über alle Messwerte iterieren 2. Den höchsten Wert ermitteln 3. Diesen ausgeben

Erwartetes Verhalten:

Höchster Messwert: 26.3°C

Hilfestellungen

Problemanalyse:

- Liste durchgehen -> for-Schleife
- Maximum suchen -> Variable für "bisher höchster Wert"
- Vergleich in jedem Durchlauf

Lösungsschritte:

1. Liste definieren
2. Variable `maximum` mit sehr kleinem Startwert initialisieren (z.B. 0 oder erster Wert)
3. for-Schleife über alle Messwerte
4. In jedem Durchlauf: Ist aktueller Wert > `maximum`?
5. Wenn ja: `maximum` aktualisieren
6. Nach Schleife: `maximum` ausgeben

Pseudocode:

```
messwerte = [23.5, 25.1, 22.8, 26.3, 24.0]  
maximum = 0 # oder messwerte[0]
```

```
FÜR JEDEN wert IN messwerte  
    WENN wert > maximum  
        maximum = wert
```

Gebe `maximum` aus

Trick für Startwert:

Sie können `maximum = messwerte[0]` setzen – dann ist der erste Wert automatisch das initiale Maximum!

9.2 Aufgabe 13: Defekte Bauteile zählen

Schwierigkeit: (Einfach)

Aufgabenstellung

Eine Qualitätskontrolle hat Bauteile mit "OK" oder "DEFEKT" markiert. Zählen Sie die defekten Bauteile.

Gegeben ist:

```
qualitaet = ["OK", "OK", "DEFEKT", "OK", "DEFEKT", "OK", "OK", "DEFEKT"]
```

Das Programm soll: 1. Über die Liste iterieren 2. Defekte Bauteile zählen 3. Die Anzahl und den Prozentsatz ausgeben

Erwartetes Verhalten:

Defekte Bauteile: 3 von 8 (37.5%)

Hilfestellungen**Problemanalyse:**

- Liste durchgehen -> for-Schleife
- Zählen -> Zähler-Variable
- Prozent berechnen -> $(\text{defekt} / \text{gesamt}) * 100$

Lösungsschritte:

1. Liste definieren
2. Zähler `defekte = 0` initialisieren
3. for-Schleife über alle Einträge
4. In jedem Durchlauf: Ist Status "DEFEKT"?
5. Wenn ja: Zähler erhöhen
6. Nach Schleife: Prozent berechnen
7. Ergebnis ausgeben

Pseudocode:

```
qualitaet = ["OK", "OK", "DEFEKT", ...]  
defekte = 0
```

```
FÜR JEDEN status IN qualitaet  
    WENN status == "DEFEKT"  
        defekte = defekte + 1
```

```
gesamt = Länge von qualitaet  
prozent = (defekte / gesamt) * 100
```

Gebe `defekte`, `gesamt` und `prozent` aus

9.3 Aufgabe 14: Lagerbericht erstellen

Schwierigkeit: (Mittel)

Aufgabenstellung

Ein Lager verwaltet Produkte in einem Dictionary. Erstellen Sie einen Bericht über den Gesamtwert.

Gegeben ist:

```
lager = {
    "Schrauben": 150,
    "Muttern": 200,
    "Bolzen": 80,
    "Dübel": 120
}

preise = {
    "Schrauben": 0.05,
    "Muttern": 0.03,
    "Bolzen": 0.15,
    "Dübel": 0.08
}
```

Das Programm soll: 1. Über alle Produkte iterieren 2. Für jedes Produkt: Bestand x Preis berechnen 3. Den Gesamtwert aller Produkte berechnen 4. Einen Bericht ausgeben

Erwartetes Verhalten:

```
=== LAGERBERICHT ===
Schrauben: 150 Stück à 0.05€ = 7.50€
Muttern: 200 Stück à 0.03€ = 6.00€
Bolzen: 80 Stück à 0.15€ = 12.00€
Dübel: 120 Stück à 0.08€ = 9.60€
=====
Gesamtwert: 35.10€
```

Hilfestellungen**Problemanalyse:**

- Zwei Dictionaries: Bestand und Preise
- Über Produkte iterieren -> `for produkt, anzahl in lager.items()`
- Gesamtwert akkumulieren -> Variable

Lösungsschritte:

1. Beide Dictionaries definieren
2. Variable `gesamtwert = 0` initialisieren
3. for-Schleife über `lager.items()` (gibt Schlüssel UND Wert)
4. In jedem Durchlauf:
 - Preis aus `preise` Dictionary holen
 - Wert berechnen: `anzahl * preis`
 - Zu `gesamtwert` addieren
 - Zeile ausgeben
5. Nach Schleife: Gesamtwert ausgeben

Pseudocode:

```
lager = {...}
preise = {...}
gesamtwert = 0
```

Gebe Überschrift aus

FÜR JEDES produkt, anzahl IN lager

```
preis = preise[produkt]
wert = anzahl * preis
gesamtwert = gesamtwert + wert
```

Gebe produkt, anzahl, preis und wert aus

Gebe gesamtwert aus

Hinweis zu .items():

lager.items() gibt Paare zurück: ("Schrauben", 150), ("Mutter", 200), ... In der for-Schleife können Sie diese direkt entpacken: for produkt, anzahl in lager.items()

10 Block F – break und continue (Mittel)

Aufwand: ca. 15-20 Minuten

Erstellen Sie die Datei `block_f_break_continue.py` für die folgenden Aufgaben.

Info-Box: break und continue

break: Beendet die Schleife **sofort**

```
for element in liste:
    if bedingung:
        break # Schleife wird beendet
```

continue: Überspringt den **aktuellen** Durchlauf, Schleife läuft weiter

```
for element in liste:
    if bedingung:
        continue # Nächstes Element
# Wird nur ausgeführt, wenn bedingung False war
```

10.1 Aufgabe 15: Erste defekte Charge finden

Schwierigkeit: (Mittel)

Aufgabenstellung

Mehrere Produktionschargen werden geprüft. Das Programm soll stoppen, sobald die erste defekte Charge gefunden wird.

Gegeben ist:

```
chargen = [
    {"id": "C001", "status": "OK"},
    {"id": "C002", "status": "OK"},
    {"id": "C003", "status": "DEFEKT"},
    {"id": "C004", "status": "OK"},
    {"id": "C005", "status": "DEFEKT"}
]
```

Das Programm soll: 1. Über die Chargen iterieren 2. Jede Charge prüfen 3. Bei der ersten defekten Charge: - Meldung ausgeben - Schleife mit `break` beenden 4. Falls keine defekte Charge: "Alle Chargen in Ordnung"

Erwartetes Verhalten:

```
Prüfe Charge C001: OK
Prüfe Charge C002: OK
Prüfe Charge C003: DEFECT
[WARNUNG] Defekte Charge gefunden: C003
Prüfung beendet.
```

Hilfestellungen

Problemanalyse:

- Suche nach erstem Vorkommen -> `break` bei Fund
- Liste von Dictionaries -> Zugriff auf Keys
- Unterscheidung: Defekt gefunden oder nicht

Lösungsschritte:

1. Liste definieren
2. Variable `defekt_gefunden` = False für Tracking
3. for-Schleife über `chargen`
4. Für jede Charge:
 - ID und Status ausgeben
 - Wenn Status "DEFEKT": Meldung, `defekt_gefunden` = True, break
5. Nach Schleife: Falls nicht gefunden, "Alle OK" ausgeben

Pseudocode:

```
chargen = [...]
defekt_gefunden = False
```

```
FÜR JEDE charge IN chargen
  Gebe "Prüfe..." aus
```

```
    WENN charge["status"] == "DEFEKT"
      Gebe Warnung aus
      defekt_gefunden = True
      BEENDE SCHLEIFE (break)
```

```
WENN NICHT defekt_gefunden
  Gebe "Alle OK" aus
```

Warum break?

Ohne break würden auch C004 und C005 geprüft, obwohl wir schon bei C003 eine Defekte gefunden haben!

10.2 Aufgabe 16: Nur gültige Messwerte verarbeiten

Schwierigkeit: (Mittel)

Aufgabenstellung

Ein Sensor liefert Messwerte, aber manche sind ungültig (< 0). Diese sollen übersprungen werden.

Gegeben ist:

```
messwerte = [23.5, -1.0, 25.1, 22.8, -5.0, 26.3, 24.0]
```

Das Programm soll: 1. Über alle Messwerte iterieren 2. Ungültige Werte (< 0) mit `continue` überspringen 3. Gültige Werte zählen und deren Summe berechnen 4. Durchschnitt der gültigen Werte ausgeben

Erwartetes Verhalten:

```
Überspringe ungültigen Wert: -1.0
Überspringe ungültigen Wert: -5.0
Anzahl gültiger Werte: 5
Durchschnitt: 24.34°C
```

Hilfestellungen**Problemanalyse:**

- Bestimmte Elemente überspringen -> `continue`
- Zähler und Summe für gültige Werte
- Durchschnitt berechnen nach Schleife

Lösungsschritte:

1. Liste definieren
2. Variablen initialisieren: `anzahl_gueltig = 0, summe = 0`
3. for-Schleife über `messwerte`
4. Für jeden Wert:
 - Wenn `< 0`: Meldung ausgeben, `continue` (Rest wird übersprungen!)
 - Sonst: Zu `summe` addieren, `anzahl_gueltig` erhöhen
5. Nach Schleife: Durchschnitt berechnen und ausgeben

Pseudocode:

```
messwerte = [...]  
anzahl_gueltig = 0  
summe = 0
```

```
FÜR JEDEN wert IN messwerte
```

```
    WENN wert < 0
```

```
        Gebe "Überspringe..." aus
```

```
        ÜBERSPRINGE REST (continue)
```

```
    # Dieser Teil wird nur für gültige Werte ausgeführt
```

```
    summe = summe + wert
```

```
    anzahl_gueltig = anzahl_gueltig + 1
```

```
durchschnitt = summe / anzahl_gueltig
```

```
Gebe anzahl_gueltig und durchschnitt aus
```

Wichtig: Nach `continue` wird der Rest der Schleife übersprungen, aber die Schleife läuft mit dem nächsten Element weiter!

11 Komplexaufgaben (Block G und H)

Aufwand: ca. 30-40 Minuten

Erstellen Sie die Datei `komplex_aufgaben.py` für die folgenden Aufgaben.

Diese Aufgaben kombinieren **alle gelernten Konzepte**: Verzweigungen, Schleifen, Listen, Dictionaries.

12 Block G – Komplexaufgabe 1

12.1 Aufgabe 17: Produktionslinien-Analyse

Schwierigkeit: (Herausforderung)

Aufgabenstellung

Drei Produktionslinien liefern täglich Stückzahlen. Analysieren Sie die Daten für eine Woche.

Gegeben ist:

```
linie_a = [95, 102, 98, 105, 100, 97, 101]
linie_b = [88, 92, 90, 85, 91, 89, 87]
linie_c = [110, 108, 112, 115, 109, 111, 113]
```

Tagesziel je Linie: 100 Stück

Schreiben Sie ein Programm, das: 1. Für jede Linie: - Durchschnitt der Woche berechnet - Beste und schlechteste Tagesproduktion findet - Anzahl der Tage zählt, an denen das Ziel (100) erreicht wurde 2. Eine Gesamtauswertung ausgibt: - Welche Linie war am besten (höchster Durchschnitt)? - Welche Linie hat das Ziel am häufigsten erreicht? - Gesamtproduktion aller Linien

Erwartetes Verhalten:

```
=== ANALYSE PRODUKTIONSLINIE A ===
Durchschnitt: 99.71 Stück/Tag
Beste Tagesproduktion: 105 Stück
Schlechteste Tagesproduktion: 95 Stück
Tagesziel erreicht: 4 von 7 Tagen

=== ANALYSE PRODUKTIONSLINIE B ===
Durchschnitt: 88.86 Stück/Tag
Beste Tagesproduktion: 92 Stück
Schlechteste Tagesproduktion: 85 Stück
Tagesziel erreicht: 0 von 7 Tagen

=== ANALYSE PRODUKTIONSLINIE C ===
Durchschnitt: 111.14 Stück/Tag
Beste Tagesproduktion: 115 Stück
Schlechteste Tagesproduktion: 108 Stück
Tagesziel erreicht: 7 von 7 Tagen

=== GESAMTAUSWERTUNG ===
Beste Linie (Durchschnitt): Linie C (111.14)
Zuverlässigste Linie: Linie C (7 Tage Ziel erreicht)
Gesamtproduktion: 2098 Stück
```

Hilfestellungen**Problemanalyse:**

- Drei Listen zu analysieren -> Wiederholtes Vorgehen (evtl. Funktion!)
- Für jede Liste: Durchschnitt, max, min, Zählen mit Bedingung
- Vergleich zwischen Linien -> Variablen für beste Linie

Lösungsschritte:**Teil 1 – Analyse je Linie (für jede der 3 Listen):**

1. Durchschnitt: `sum(liste) / len(liste)`
2. Maximum: for-Schleife oder `max(liste)`
3. Minimum: for-Schleife oder `min(liste)`
4. Ziel-Tage zählen: for-Schleife mit if, Zähler erhöhen wenn `>= 100`
5. Ausgabe formatieren

Teil 2 – Gesamtauswertung:

1. Durchschnitte der drei Linien vergleichen (if-elif-else oder max mit Indizes)
2. Ziel-Tage der drei Linien vergleichen
3. Gesamtproduktion: Summe aller drei Listen

Pseudocode (vereinfacht):

```

linie_a = [...]
linie_b = [...]
linie_c = [...]
tagesziel = 100

# Für Linie A analysieren:
durchschnitt_a = Berechne Durchschnitt von linie_a
max_a = Finde Maximum in linie_a
min_a = Finde Minimum in linie_a
ziel_tage_a = 0
FÜR JEDEN wert IN linie_a
    WENN wert >= tagesziel
        ziel_tage_a erhöhen

Gebe Ergebnisse für A aus

# Wiederholen für B und C...

# Gesamtauswertung:
durchschnitte = [durchschnitt_a, durchschnitt_b, durchschnitt_c]
beste_linie = Finde Index des höchsten Durchschnitts
...

Gebe Gesamtauswertung aus

```

Tipp für Fortgeschrittene:

Sie können eine Funktion `analysiere_linie(daten, name)` schreiben, die die Analyse durchführt – dann müssen Sie den Code nicht dreimal schreiben!

13 Block H – Komplexaufgabe 2

13.1 Aufgabe 18: Qualitätsbericht-Generator

Schwierigkeit: (Herausforderung)

Aufgabenstellung

Ein automatisches Qualitätskontrollsystem erfasst Bauteile mit verschiedenen Attributen. Erstellen Sie einen umfassenden Bericht.

Gegeben ist:

```
bauteile = [
  {"id": "B001", "typ": "Schraube", "laenge": 50, "qualitaet": "OK"},
  {"id": "B002", "typ": "Mutter", "durchmesser": 8, "qualitaet": "OK"},
  {"id": "B003", "typ": "Schraube", "laenge": 45, "qualitaet": "DEFEKT"},
  {"id": "B004", "typ": "Bolzen", "laenge": 100, "qualitaet": "OK"},
  {"id": "B005", "typ": "Schraube", "laenge": 50, "qualitaet": "OK"},
  {"id": "B006", "typ": "Mutter", "durchmesser": 10, "qualitaet": "DEFEKT"},
  {"id": "B007", "typ": "Bolzen", "laenge": 95, "qualitaet": "OK"},
  {"id": "B008", "typ": "Schraube", "laenge": 48, "qualitaet": "DEFEKT"}
]
```

Schreiben Sie ein Programm, das folgende Statistiken erstellt:

1. Gesamtübersicht:

- Anzahl aller Bauteile
- Anzahl defekter Bauteile
- Fehlerquote in Prozent

2. Nach Typ gruppiert:

- Für jeden Typ (Schraube, Mutter, Bolzen):
 - Anzahl insgesamt
 - Anzahl defekt
 - Fehlerquote für diesen Typ

3. Detailliste defekter Bauteile:

- ID und Typ aller defekten Bauteile

Erwartetes Verhalten:

=== QUALITÄTSBERICHT ===

GESAMTÜBERSICHT:

Geprüfte Bauteile: 8

Defekte Bauteile: 3

Fehlerquote: 37.5%

ANALYSE NACH TYP:

Typ: Schraube

Gesamt: 4 | Defekt: 2 | Fehlerquote: 50.0%

Typ: Mutter

Gesamt: 2 | Defekt: 1 | Fehlerquote: 50.0%

Typ: Bolzen

Gesamt: 2 | Defekt: 0 | Fehlerquote: 0.0%

DEFEKTE BAUTEILE:

B003 – Schraube

B006 – Mutter

B008 – Schraube

Hilfestellungen**Problemanalyse:**

- Liste von Dictionaries
- Mehrere Durchläufe über die Liste (oder geschickt kombinieren)
- Gruppierung nach Typ -> Dictionary als Hilfsdatenstruktur
- Verschiedene Zählvorgänge

Lösungsschritte:**Teil 1 – Gesamtübersicht:**

1. Gesamtanzahl: `len(bauteile)`
2. Defekte zählen: `for`-Schleife, `if qualitaet == "DEFEKT"`, Zähler
3. Fehlerquote berechnen

Teil 2 – Nach Typ gruppiert:

1. Alle Typen ermitteln (Set oder manuelle Liste: "Schraube", "Mutter", "Bolzen")
2. Für jeden Typ:
 - Liste durchgehen
 - Zählen: Wie viele haben diesen Typ?
 - Zählen: Wie viele davon sind defekt?
 - Fehlerquote für Typ berechnen
 - Ausgeben

Teil 3 – Defekte Bauteile:

1. Liste durchgehen
2. Wenn `qualitaet == "DEFEKT"`: ID und Typ ausgeben

Pseudocode (Teil 2 als Beispiel):

```
typen = ["Schraube", "Mutter", "Bolzen"]
```

```
FÜR JEDEN typ IN typen
    gesamt_typ = 0
    defekt_typ = 0
```

```
    FÜR JEDES bauteil IN bauteile
        WENN bauteil["typ"] == typ
            gesamt_typ erhöhen
```

```
        WENN bauteil["qualitaet"] == "DEFEKT"
            defekt_typ erhöhen
```

```
    fehlerquote_typ = (defekt_typ / gesamt_typ) * 100
    Gebe Statistik für typ aus
```

Hinweis zu verschachtelten Schleifen:

Bei Teil 2 haben wir eine Schleife IN einer Schleife:

- Äußere Schleife: Über Typen
- Innere Schleife: Über alle Bauteile

Das ist erlaubt und oft notwendig!

Alternative mit Dictionary:

Sie könnten auch ein Dictionary `typ_stats = {}` anlegen und in **einem** Durchlauf alle Informationen sammeln – das ist effizienter, aber komplexer!

14 Musterlösungen

Lösungen Block A – if und if-else

Lösung Aufgabe 1: Produktionsüberwachung

```
# Aufgabe 1: Produktionsüberwachung

# Tagesziel definieren
tagesziel = 100

# Produzierte Stückzahl vom Benutzer abfragen
produziert = int(input("Produzierte Stückzahl: "))

# Prüfen, ob Tagesziel erreicht wurde
if produziert >= tagesziel:
    print("Tagesziel erreicht!")
else:
    print("Tagesziel nicht erreicht.")
```

Erklärung:

- Wir verwenden `int()`, um die Benutzereingabe in eine Ganzzahl zu konvertieren
- Die Bedingung `produziert >= tagesziel` prüft, ob das Ziel erreicht oder übertroffen wurde
- `if-else` garantiert, dass immer eine der beiden Meldungen ausgegeben wird

Verwendete Konzepte:

- `if-else`-Anweisung
 - Vergleichsoperator `>=`
 - `input()` und Typkonvertierung mit `int()`
-

Lösung Aufgabe 2: Temperaturüberwachung

```
# Aufgabe 2: Temperaturüberwachung

# Grenzwert definieren
grenzwert = 80

# Aktuelle Temperatur abfragen
temperatur = float(input("Aktuelle Temperatur (°C): "))

# Prüfen, ob Grenzwert überschritten
if temperatur > grenzwert:
    print("[WARNUNG] WARNUNG: Überhitzung! Maschine prüfen!")
else:
    print("Temperatur im Normbereich.")
```

Erklärung:

- `float()` wird verwendet, da Temperaturen Dezimalzahlen sein können
- Der Vergleichsoperator `>` (nicht `>=`) prüft auf strikte Überschreitung
- Bei genau 80°C ist noch alles im Normbereich (daher `>` statt `>=`)

Verwendete Konzepte:

- `if-else`-Anweisung
 - Vergleichsoperator `>`
 - `float()` für Dezimalzahlen
-

Lösung Aufgabe 3: Bauteil-ID Klassifizierung


```
# Aufgabe 3: Bauteil-ID Klassifizierung

# Bauteil-ID abfragen
bauteil_id = int(input("Bauteil-ID: "))

# Prüfen, ob ID gerade oder ungerade ist
# Modulo 2 gibt 0 für gerade Zahlen, 1 für ungerade
if bauteil_id % 2 == 0:
    print("Lager A (gerade ID)")
else:
    print("Lager B (ungerade ID)")
```

Erklärung:

- Der Modulo-Operator % gibt den Rest einer Division zurück
- $\text{zahl} \% 2$ ergibt 0, wenn die Zahl gerade ist (rest-freie Division durch 2)
- $\text{zahl} \% 2$ ergibt 1, wenn die Zahl ungerade ist (Rest 1 bei Division durch 2)
- Beispiele: $1042 \% 2 = 0$ (gerade), $1043 \% 2 = 1$ (ungerade)

Verwendete Konzepte:

- if-else-Anweisung
- Modulo-Operator %
- Vergleichsoperator ==

Lösungen Block B – elif**Lösung Aufgabe 4: Qualitätskontrolle***# Aufgabe 4: Qualitätskontrolle**# Eingaben abfragen*produzierte = `int(input("Produzierte Teile: "))`fehlerhafte = `int(input("Fehlerhafte Teile: "))`*# Fehlerquote berechnen*fehlerquote = `(fehlerhafte / produzierte) * 100`*# Fehlerquote ausgeben*`print(f"Fehlerquote: {fehlerquote:.1f}%")`*# Bewertung ausgeben (von klein nach groß prüfen!)*`if fehlerquote < 1:` `print("Bewertung: Exzellente Qualität")``elif fehlerquote < 3:` `print("Bewertung: Gute Qualität")``elif fehlerquote < 5:` `print("Bewertung: Akzeptable Qualität - Optimierung empfohlen")``else:` `print("Bewertung: Kritisch - Sofortige Maßnahmen erforderlich!")`**Erklärung:**

- Die Division `fehlerhafte / produzierte` ergibt einen Dezimalbruch (z.B. 0.015 für 1.5%)
- Multiplikation mit 100 wandelt in Prozent um
- F-String `:.1f` formatiert auf 1 Nachkommastelle
- **Wichtig:** Bedingungen von klein nach groß, da Python beim ersten Treffer stoppt

Verwendete Konzepte:

- if-elif-else-Kette
- Bereichsprüfung mit `<` Operator
- Arithmetische Operatoren (`/`, `*`)
- F-String-Formatierung

Lösung Aufgabe 5: Maschinendrehzahl klassifizieren*# Aufgabe 5: Maschinendrehzahl klassifizieren**# Drehzahl abfragen*drehzahl = `int(input("Aktuelle Drehzahl (U/min): "))`*# Betriebsmodus bestimmen*`if drehzahl < 500:` `print("Betriebsmodus: Standby-Modus")``elif drehzahl < 1500:` `print("Betriebsmodus: Niedrige Drehzahl")``elif drehzahl < 3000:` `print("Betriebsmodus: Normal-Betrieb")``else:` `print("Betriebsmodus: Hochleistungsmodus")`**Erklärung:**

- Vier Bereiche werden abgebildet durch drei elif-Bedingungen + else
- Reihenfolge von klein nach groß ist entscheidend:
 - Erste Prüfung: `< 500` (Standby)

- Wenn nicht: < 1500 (bedeutet automatisch ≥ 500 , also "Niedrige Drehzahl")
- Wenn nicht: < 3000 (bedeutet ≥ 1500 , also "Normal-Betrieb")
- Sonst: ≥ 3000 (Hochleistung)

Verwendete Konzepte:

- if-elif-elif-else-Struktur
- Bereichsprüfung
- Implizite Unter- und Obergrenzen durch Reihenfolge

Lösung Aufgabe 6: Lagerverwaltung – Füllstand prüfen

Aufgabe 6: Lagerverwaltung – Füllstand prüfen

Kapazität als Konstante

kapazitaet = 1000

Aktuellen Bestand abfragen

bestand = int(input("Aktueller Bestand: "))

Füllstand in Prozent berechnen

fuellstand = (bestand / kapazitaet) * 100

Füllstand ausgeben

print(f"Füllstand: {fuellstand:.1f}%")

Status ausgeben

if fuellstand < 25:

 print("Status: Kritisch niedrig - Nachbestellung erforderlich!")

elif fuellstand < 50:

 print("Status: Niedrig - Bald nachbestellen")

elif fuellstand < 90:

 print("Status: Normaler Füllstand")

else:

 print("Status: Fast voll - Platzmangel prüfen")

Erklärung:

- Die Kapazität (1000) ist fest und wird als Konstante definiert
- Prozentberechnung: (teil / ganzes) * 100
- Vier Füllstandsbereiche mit drei Grenzen (25%, 50%, 90%)
- Jeder Bereich hat eine spezifische Handlungsempfehlung

Verwendete Konzepte:

- if-elif-elif-else
- Prozentberechnung
- F-String mit Formatierung
- Konstanten (kapazitaet)

Lösungen Block C – match-case

Lösung Aufgabe 7: Maschinentyp-Auswahl

```
# Aufgabe 7: Maschinentyp-Auswahl

# Maschinentyp abfragen und in Großbuchstaben umwandeln
typ = input("Maschinentyp (A-E): ").upper()

# Mit match-case die Eigenschaften ausgeben
match typ:
    case "A":
        print("CNC-Fräse - Maximale Genauigkeit: 0.01mm")
    case "B":
        print("Drehmaschine - Maximale Drehzahl: 3000 U/min")
    case "C":
        print("Laserschneider - Maximale Leistung: 2000W")
    case "D":
        print("3D-Drucker - Druckvolumen: 300x300x400mm")
    case "E":
        print("Schweißroboter - Reichweite: 2.5m")
    case _:
        print("Unbekannter Maschinentyp")
```

Erklärung:

- `.upper()` wandelt die Eingabe in Großbuchstaben um ("a" -> "A")
 - So funktioniert die Prüfung unabhängig von Groß-/Kleinschreibung
- `match typ:` vergleicht die Variable `typ` mit den folgenden cases
- `case _:` ist der Standard-Fall (wie `else` bei `if-elif`)
- Jeder case endet mit `:` und der Code darunter ist eingerückt
- Sobald ein case passt, wird nur dieser ausgeführt (kein `break` nötig!)

Verwendete Konzepte:

- match-case-Struktur (Python 3.10+)
 - String-Methode `.upper()`
 - Standard-Fall mit `_`
-

Lösung Aufgabe 8: Schichtplanung

```
# Aufgabe 8: Schichtplanung

# Schichtnummer abfragen
schicht = int(input("Schichtnummer (1-3): "))

# Mit match-case Schichtinformationen ausgeben
match schicht:
    case 1:
        print("Frühschicht: 06:00 - 14:00 Uhr, Zuschlag: 0%")
    case 2:
        print("Spätschicht: 14:00 - 22:00 Uhr, Zuschlag: 10%")
    case 3:
        print("Nachtschicht: 22:00 - 06:00 Uhr, Zuschlag: 25%")
    case _:
        print("Ungültige Schichtnummer")
```

Erklärung:

- match-case funktioniert auch mit Zahlen, nicht nur mit Strings
- Die Schichtnummer wird als `int` eingegeben, daher Vergleich mit 1, 2, 3 (ohne Anführungszeichen)
- Jede Schicht hat spezifische Zeiten und einen Lohnzuschlag

- Der Standard-Fall `case _`: fängt alle ungültigen Eingaben ab (z.B. 0, 4, 99, ...)

Verwendete Konzepte:

- match-case mit Zahlen
- int-Konvertierung
- Standard-Fall für Fehlerbehandlung

Lösungen Block D – while-Schleifen

Lösung Aufgabe 9: Countdown für Maschinenstart

```
# Aufgabe 9: Countdown für Maschinenstart

# Startwert für Countdown
countdown = 10

# Countdown von 10 bis 1
while countdown >= 1:
    print(countdown)
    countdown = countdown - 1 # oder: countdown -= 1

# Nach dem Countdown
print("START!")
```

Erklärung:

- Die Variable `countdown` startet bei 10
- Die Bedingung `countdown >= 1` ist wahr, solange `countdown` 1 oder größer ist
- In jedem Durchlauf:
 1. Aktuelle Zahl wird ausgegeben
 2. `countdown` wird um 1 verringert
- Wenn `countdown` auf 0 sinkt, ist die Bedingung falsch -> Schleife endet
- "START!" wird nach der Schleife ausgegeben

Wichtig: Die Zeile `countdown = countdown - 1` ist entscheidend! Ohne sie würde `countdown` immer 10 bleiben -> Endlosschleife!

Verwendete Konzepte:

- while-Schleife
- Vergleichsoperator `>=`
- Dekrementierung (Verringerung) einer Variable

Lösung Aufgabe 10: Eingabe-Validierung für Temperatur

```
# Aufgabe 10: Eingabe-Validierung für Temperatur

# Endlosschleife für wiederholte Eingabe
while True:
    # Temperatur abfragen
    temperatur = float(input("Temperatur (0-200°C): "))

    # Prüfen, ob Temperatur im gültigen Bereich
    if temperatur >= 0 and temperatur <= 200:
        # Gültig: Bestätigung und Schleife beenden
        print(f"Temperatur akzeptiert: {temperatur}°C")
        break # Schleife wird hier beendet!
    else:
        # Ungültig: Fehlermeldung, Schleife läuft weiter
        print("Ungültige Eingabe! Temperatur muss zwischen 0 und 200°C liegen.")
```

Erklärung:

- `while True:` startet eine Endlosschleife (True ist immer wahr)
- In jedem Durchlauf wird nach einer Temperatur gefragt
- Die Bedingung prüft mit `and`: Beide Teile müssen wahr sein
 - `temperatur >= 0` UND `temperatur <= 200`
 - Alternative kürzere Schreibweise: `0 <= temperatur <= 200`
- Bei gültiger Eingabe: `break` beendet die Schleife sofort

- Bei ungültiger Eingabe: Fehlermeldung, dann läuft die Schleife erneut

Warum while True?

Wir wissen nicht im Voraus, wie oft der Benutzer eine falsche Eingabe macht. Daher: Endlosschleife mit explizitem Abbruch durch `break`.

Verwendete Konzepte:

- while True Schleife
- break-Statement
- Logischer Operator and
- Bereichsprüfung

Lösung Aufgabe 11: Produktionsdaten sammeln

```
# Aufgabe 11: Produktionsdaten sammeln

# Gesamtsumme initialisieren
gesamtstueckzahl = 0

# Endlosschleife für Eingaben
while True:
    # Produktnamen abfragen
    produkt = input("Produktname (oder STOP): ")

    # Prüfen, ob Benutzer stoppen möchte (unabhängig von Groß-/Kleinschreibung)
    if produkt.upper() == "STOP":
        break # Schleife beenden

    # Stückzahl für dieses Produkt abfragen
    stueckzahl = int(input("Stückzahl: "))

    # Zur Gesamtsumme addieren
    gesamtstueckzahl = gesamtstueckzahl + stueckzahl

# Nach Schleife: Gesamtergebnis ausgeben
print(f"\nGesamtzahl produzierte Teile: {gesamtstueckzahl}")
```

Erklärung:

- `gesamtstueckzahl` akkumuliert (sammelt) alle eingegebenen Stückzahlen
- `.upper()` wandelt die Eingabe in Großbuchstaben um
 - “stop”, “Stop”, “STOP” werden alle zu “STOP”
 - Dann funktioniert der Vergleich `== "STOP"` immer
- Bei “STOP” wird die Schleife sofort mit `break` beendet
 - Die Zeilen danach (Stückzahl-Abfrage) werden **nicht** mehr ausgeführt
- Bei jedem Produkt wird die Stückzahl zur Gesamtsumme addiert

Akkumulations-Muster:

```
summe = 0
while bedingung:
    wert = # ... Eingabe oder Berechnung
    summe = summe + wert
```

Dieses Muster begegnet Ihnen sehr häufig!

Verwendete Konzepte:

- while True mit break
- String-Methode `.upper()`
- Akkumulation (Summenbildung)

- Zwei verschiedene Eingaben in der Schleife

Lösungen Block E – for-Schleifen**Lösung Aufgabe 12: Maximum aus Messwerten finden**

```
# Aufgabe 12: Maximum aus Messwerten finden

# Liste der Messwerte
messwerte = [23.5, 25.1, 22.8, 26.3, 24.0]

# Maximum mit dem ersten Wert initialisieren
maximum = messwerte[0] # Startwert: 23.5

# Über alle Messwerte iterieren
for wert in messwerte:
    # Wenn aktueller Wert größer als bisheriges Maximum
    if wert > maximum:
        maximum = wert # Neues Maximum gefunden

# Ergebnis ausgeben
print(f"Höchster Messwert: {maximum}°C")
```

Erklärung:

- Wir initialisieren `maximum` mit dem ersten Wert der Liste
 - Alternative: `maximum = 0`, aber was, wenn alle Werte negativ wären?
- Die `for`-Schleife geht durch jeden Wert:
 - **Durchlauf 1:** `wert = 23.5`, ist `23.5 > 23.5`? Nein -> `maximum` bleibt `23.5`
 - **Durchlauf 2:** `wert = 25.1`, ist `25.1 > 23.5`? Ja -> `maximum` wird `25.1`
 - **Durchlauf 3:** `wert = 22.8`, ist `22.8 > 25.1`? Nein -> `maximum` bleibt `25.1`
 - **Durchlauf 4:** `wert = 26.3`, ist `26.3 > 25.1`? Ja -> `maximum` wird `26.3`
 - **Durchlauf 5:** `wert = 24.0`, ist `24.0 > 26.3`? Nein -> `maximum` bleibt `26.3`
- Endergebnis: `maximum = 26.3`

Verwendete Konzepte:

- `for`-Schleife über Liste
- `if`-Anweisung innerhalb der Schleife
- Variable zur Speicherung des bisherigen Maximums

Hinweis: Python hat eine eingebaute `max()`-Funktion: `maximum = max(messwerte)` Aber es ist wichtig, das Prinzip zu verstehen!

Lösung Aufgabe 13: Defekte Bauteile zählen

```
# Aufgabe 13: Defekte Bauteile zählen

# Liste mit Qualitätsstatus
qualitaet = ["OK", "OK", "DEFEKT", "OK", "DEFEKT", "OK", "OK", "DEFEKT"]

# Zähler für defekte Bauteile
defekte = 0

# Über alle Einträge iterieren
for status in qualitaet:
    # Wenn Status "DEFEKT" ist, Zähler erhöhen
    if status == "DEFEKT":
        defekte = defekte + 1 # oder: defekte += 1

# Gesamtanzahl
gesamt = len(qualitaet)
```

```
# Prozentsatz berechnen
prozent = (defekte / gesamt) * 100

# Ergebnis ausgeben
print(f"Defekte Bauteile: {defekte} von {gesamt} ({prozent:.1f}%")
```

Erklärung:

- Zählermuster: Variable defekte startet bei 0
- In jedem Durchlauf wird geprüft, ob der Status "DEFEKT" ist
- Falls ja: Zähler um 1 erhöhen
- Nach der Schleife:
 - Gesamtanzahl mit len() ermitteln
 - Prozentsatz berechnen: (teil / ganzes) * 100
 - Formatiert ausgeben

Zählermuster (sehr häufig!):

```
zaehler = 0
for element in liste:
    if bedingung:
        zaehler = zaehler + 1
```

Verwendete Konzepte:

- for-Schleife über Liste
- Zähler-Variable
- Bedingung innerhalb der Schleife
- len() für Listenlänge
- Prozentberechnung

Lösung Aufgabe 14: Lagerbericht erstellen

```
# Aufgabe 14: Lagerbericht erstellen

# Dictionaries mit Beständen und Preisen
lager = {
    "Schrauben": 150,
    "Muttern": 200,
    "Bolzen": 80,
    "Dübel": 120
}

preise = {
    "Schrauben": 0.05,
    "Muttern": 0.03,
    "Bolzen": 0.15,
    "Dübel": 0.08
}

# Gesamtwert-Variable initialisieren
gesamtwert = 0.0

# Überschrift ausgeben
print("=== LAGERBERICHT ===")

# Über alle Produkte im Lager iterieren
for produkt, anzahl in lager.items():
    # Preis für dieses Produkt aus dem preise-Dictionary holen
    preis = preise[produkt]
```

```
# Wert für dieses Produkt berechnen
wert = anzahl * preis

# Zum Gesamtwert addieren
gesamtwert = gesamtwert + wert

# Zeile für dieses Produkt ausgeben
print(f"{produkt}: {anzahl} Stück à {preis:.2f}€ = {wert:.2f}€")

# Abschluss
print("=" * 20)
print(f"Gesamtwert: {gesamtwert:.2f}€")
```

Erklärung:

- `.items()` bei Dictionaries gibt Paare zurück: (schlüssel, wert)
- `for produkt, anzahl in lager.items():` entpackt diese Paare direkt
 - `produkt` enthält den Schlüssel (z.B. "Schrauben")
 - `anzahl` enthält den Wert (z.B. 150)
- Mit `preise[produkt]` holen wir den Preis aus dem anderen Dictionary
- Berechnung: `wert = anzahl * preis`
- Akkumulation: `gesamtwert += wert`

Zwei Dictionaries verknüpfen:

Beide haben dieselben Schlüssel -> wir können sie über den Schlüssel verbinden!

F-String Formatierung:

- `:.2f` bedeutet: Dezimalzahl mit 2 Nachkommastellen
- `"=" * 20` erzeugt einen String mit 20 Gleichheitszeichen

Verwendete Konzepte:

- for-Schleife über Dictionary mit `.items()`
- Tuple-Entpackung (`produkt, anzahl`)
- Dictionary-Zugriff mit Schlüssel
- Akkumulation
- F-String mit Formatierung

Lösungen Block F – break und continue

Lösung Aufgabe 15: Erste defekte Charge finden

Aufgabe 15: Erste defekte Charge finden

Liste der Chargen

```
chargen = [
    {"id": "C001", "status": "OK"},
    {"id": "C002", "status": "OK"},
    {"id": "C003", "status": "DEFEKT"},
    {"id": "C004", "status": "OK"},
    {"id": "C005", "status": "DEFEKT"}
]
```

Variable zum Tracking, ob defekte Charge gefunden wurde
defekt_gefunden = False

Über alle Chargen iterieren

```
for charge in chargen:
    # Charge ID und Status ausgeben
    print(f"Prüfe Charge {charge['id']}: {charge['status']}")

    # Wenn defekte Charge gefunden
    if charge["status"] == "DEFEKT":
        print(f"[WARNUNG] Defekte Charge gefunden: {charge['id']}")
        defekt_gefunden = True
        break # Schleife sofort beenden!
```

```
print("Prüfung beendet.")
```

Falls keine defekte Charge gefunden wurde

```
if not defekt_gefunden:
    print("[OK] Alle Chargen in Ordnung")
```

Erklärung:

- Liste von Dictionaries: Jede Charge ist ein Dictionary mit "id" und "status"
- Zugriff auf Dictionary-Werte: charge["id"] und charge["status"]
- defekt_gefunden ist ein Boolean-Flag (Merker)
 - Startet als False
 - Wird auf True gesetzt, wenn Defekt gefunden
- break beendet die Schleife sofort:
 - C004 und C005 werden **nicht mehr** geprüft
 - Nur die erste defekte Charge wird gefunden

Ohne break:

```
Prüfe Charge C001: OK
Prüfe Charge C002: OK
Prüfe Charge C003: DEFEKT
[WARNUNG] Defekte Charge gefunden: C003
Prüfe Charge C004: OK      ← würde weiterlaufen
Prüfe Charge C005: DEFEKT  ← würde auch gemeldet
```

Mit break:

Die Schleife stoppt bei C003 -> effizienter!

Verwendete Konzepte:

- for-Schleife über Liste von Dictionaries
- Dictionary-Zugriff

- Boolean-Variable als Flag
- break-Statement
- Negation mit not

Lösung Aufgabe 16: Nur gültige Messwerte verarbeiten

```
# Aufgabe 16: Nur gültige Messwerte verarbeiten

# Liste der Messwerte (inkl. ungültiger Werte < 0)
messwerte = [23.5, -1.0, 25.1, 22.8, -5.0, 26.3, 24.0]

# Variablen für gültige Werte
anzahl_gueltig = 0
summe = 0.0

# Über alle Messwerte iterieren
for wert in messwerte:
    # Ungültige Werte überspringen
    if wert < 0:
        print(f"Überspringe ungültigen Wert: {wert}")
        continue # Rest wird übersprungen, nächster Durchlauf!

    # Ab hier: Code wird nur für gültige Werte ausgeführt
    summe = summe + wert
    anzahl_gueltig = anzahl_gueltig + 1

# Durchschnitt berechnen
durchschnitt = summe / anzahl_gueltig

# Ergebnis ausgeben
print(f"Anzahl gültiger Werte: {anzahl_gueltig}")
print(f"Durchschnitt: {durchschnitt:.2f}°C")
```

Erklärung:

- continue überspringt den **Rest des aktuellen Durchlaufs**
- Ablauf für wert = -1.0:
 1. Bedingung wert < 0 ist wahr
 2. "Überspringe..." wird ausgegeben
 3. continue wird ausgeführt
 4. Die Zeilen summe = summe + wert und anzahl_gueltig += 1 werden **NICHT** ausgeführt
 5. Schleife macht mit nächstem Wert weiter (25.1)
- Ablauf für wert = 23.5:
 1. Bedingung wert < 0 ist falsch
 2. continue wird **nicht** ausgeführt
 3. Wert wird zur Summe addiert
 4. Zähler wird erhöht

Unterschied zu break:

- break: Beendet die **gesamte** Schleife
- continue: Überspringt nur den **aktuellen** Durchlauf

Verwendete Konzepte:

- for-Schleife über Liste
- continue-Statement
- Zähler und Akkumulator
- Division für Durchschnitt

Lösungen Block G – Komplexaufgabe 1**Lösung Aufgabe 17: Produktionslinien-Analyse***# Aufgabe 17: Produktionslinien-Analyse**# Produktionsdaten für drei Linien (eine Woche, 7 Tage)*

linie_a = [95, 102, 98, 105, 100, 97, 101]

linie_b = [88, 92, 90, 85, 91, 89, 87]

linie_c = [110, 108, 112, 115, 109, 111, 113]

Tagesziel

tagesziel = 100

=== ANALYSE LINIE A ===

print("=== ANALYSE PRODUKTIONSLINIE A ===")

Durchschnitt berechnen

durchschnitt_a = sum(linie_a) / len(linie_a)

print(f"Durchschnitt: {durchschnitt_a:.2f} Stück/Tag")

Beste und schlechteste Tagesproduktion

beste_a = max(linie_a)

schlechteste_a = min(linie_a)

print(f"Beste Tagesproduktion: {beste_a} Stück")

print(f"Schlechteste Tagesproduktion: {schlechteste_a} Stück")

Tage mit Zielerreichung zählen

ziel_tage_a = 0

for produktion in linie_a:

if produktion >= tagesziel:

ziel_tage_a += 1

print(f"Tagesziel erreicht: {ziel_tage_a} von {len(linie_a)} Tagen")

print()

=== ANALYSE LINIE B ===

print("=== ANALYSE PRODUKTIONSLINIE B ===")

durchschnitt_b = sum(linie_b) / len(linie_b)

print(f"Durchschnitt: {durchschnitt_b:.2f} Stück/Tag")

beste_b = max(linie_b)

schlechteste_b = min(linie_b)

print(f"Beste Tagesproduktion: {beste_b} Stück")

print(f"Schlechteste Tagesproduktion: {schlechteste_b} Stück")

ziel_tage_b = 0

for produktion in linie_b:

if produktion >= tagesziel:

ziel_tage_b += 1

print(f"Tagesziel erreicht: {ziel_tage_b} von {len(linie_b)} Tagen")

print()

=== ANALYSE LINIE C ===

print("=== ANALYSE PRODUKTIONSLINIE C ===")

durchschnitt_c = sum(linie_c) / len(linie_c)

```

print(f"Durchschnitt: {durchschnitt_c:.2f} Stück/Tag")

beste_c = max(linie_c)
schlechteste_c = min(linie_c)
print(f"Beste Tagesproduktion: {beste_c} Stück")
print(f"Schlechteste Tagesproduktion: {schlechteste_c} Stück")

ziel_tage_c = 0
for produktion in linie_c:
    if produktion >= tagesziel:
        ziel_tage_c += 1

print(f"Tagesziel erreicht: {ziel_tage_c} von {len(linie_c)} Tagen")
print()

# === GESAMTAUSWERTUNG ===
print("=== GESAMTAUSWERTUNG ===")

# Beste Linie nach Durchschnitt
if durchschnitt_a > durchschnitt_b and durchschnitt_a > durchschnitt_c:
    beste_linie = "Linie A"
    bester_schnitt = durchschnitt_a
elif durchschnitt_b > durchschnitt_c:
    beste_linie = "Linie B"
    bester_schnitt = durchschnitt_b
else:
    beste_linie = "Linie C"
    bester_schnitt = durchschnitt_c

print(f"Beste Linie (Durchschnitt): {beste_linie} ({bester_schnitt:.2f})")

# Zuverlässigste Linie (meiste Zielerreichungen)
if ziel_tage_a > ziel_tage_b and ziel_tage_a > ziel_tage_c:
    zuverlaessigste = "Linie A"
    max_ziel_tage = ziel_tage_a
elif ziel_tage_b > ziel_tage_c:
    zuverlaessigste = "Linie B"
    max_ziel_tage = ziel_tage_b
else:
    zuverlaessigste = "Linie C"
    max_ziel_tage = ziel_tage_c

print(f"Zuverlässigste Linie: {zuverlaessigste} ({max_ziel_tage} Tage Ziel erreicht)")

# Gesamtproduktion
gesamtproduktion = sum(linie_a) + sum(linie_b) + sum(linie_c)
print(f"Gesamtproduktion: {gesamtproduktion} Stück")

```

Erklärung:

- **Wiederholtes Muster:** Die Analyse für jede Linie folgt demselben Schema
 - In der Praxis würde man eine Funktion schreiben, um Code-Duplikation zu vermeiden
- **Verwendete Built-in-Funktionen:**
 - `sum()`: Summiert alle Elemente einer Liste
 - `max()`: Findet das Maximum
 - `min()`: Findet das Minimum
 - `len()`: Anzahl der Elemente
- **Zählen mit Bedingung:** for-Schleife + if-Abfrage + Zähler-Erhöhung
- **Vergleich von drei Werten:** Verschachtelte if-elif-else

Verwendete Konzepte:

- Mehrere Listen
- for-Schleifen mit Bedingung
- Vergleichsoperatoren
- Logische Operatoren (and)
- Built-in-Funktionen
- Verschachtelte Verzweigungen

Lösungen Block H – Komplexaufgabe 2**Lösung Aufgabe 18: Qualitätsbericht-Generator**

Aufgabe 18: Qualitätsbericht-Generator

Liste der Bauteile

```
bauteile = [
    {"id": "B001", "typ": "Schraube", "laenge": 50, "qualitaet": "OK"},
    {"id": "B002", "typ": "Mutter", "durchmesser": 8, "qualitaet": "OK"},
    {"id": "B003", "typ": "Schraube", "laenge": 45, "qualitaet": "DEFEKT"},
    {"id": "B004", "typ": "Bolzen", "laenge": 100, "qualitaet": "OK"},
    {"id": "B005", "typ": "Schraube", "laenge": 50, "qualitaet": "OK"},
    {"id": "B006", "typ": "Mutter", "durchmesser": 10, "qualitaet": "DEFEKT"},
    {"id": "B007", "typ": "Bolzen", "laenge": 95, "qualitaet": "OK"},
    {"id": "B008", "typ": "Schraube", "laenge": 48, "qualitaet": "DEFEKT"}
]
```

```
print("=== QUALITÄTSBERICHT ===\n")
```

=== TEIL 1: GESAMTÜBERSICHT ===

```
print("GESAMTÜBERSICHT:")
```

Gesamtanzahl

```
gesamt = len(bauteile)
print(f"Geprüfte Bauteile: {gesamt}")
```

Defekte zählen

```
anzahl_defekt = 0
for bauteil in bauteile:
    if bauteil["qualitaet"] == "DEFEKT":
        anzahl_defekt += 1
```

Fehlerquote berechnen

```
fehlerquote = (anzahl_defekt / gesamt) * 100
```

```
print(f"Defekte Bauteile: {anzahl_defekt}")
```

```
print(f"Fehlerquote: {fehlerquote:.1f}%\n")
```

=== TEIL 2: ANALYSE NACH TYP ===

```
print("ANALYSE NACH TYP:")
```

Liste der Typen (alternativ: set(bauteil["typ"] for bauteil in bauteile))

```
typen = ["Schraube", "Mutter", "Bolzen"]
```

Für jeden Typ Statistiken erstellen

```
for typ in typen:
    # Zähler für diesen Typ
    gesamt_typ = 0
```



```

defekt_typ = 0

# Über alle Bauteile iterieren
for bauteil in bauteile:
    # Wenn Bauteil von diesem Typ ist
    if bauteil["typ"] == typ:
        gesamt_typ += 1

    # Wenn zusätzlich defekt
    if bauteil["qualitaet"] == "DEFEKT":
        defekt_typ += 1

# Fehlerquote für diesen Typ
if gesamt_typ > 0: # Division durch 0 vermeiden
    fehlerquote_typ = (defekt_typ / gesamt_typ) * 100
else:
    fehlerquote_typ = 0

# Ausgabe für diesen Typ
print(f"Typ: {typ}")
print(f"  Gesamt: {gesamt_typ} | Defekt: {defekt_typ} | Fehlerquote: {fehlerquote_typ:.1f}%\n")

# === TEIL 3: DETAILLISTE DEFEKTER BAUTEILE ===
print("DEFEKTE BAUTEILE:")

for bauteil in bauteile:
    if bauteil["qualitaet"] == "DEFEKT":
        print(f"  {bauteil['id']} - {bauteil['typ']}")

```

Erklärung:

Teil 1 – Gesamtübersicht:

- Einfache Zählung mit for-Schleife und if
- Fehlerquote aus Verhältnis berechnen

Teil 2 – Nach Typ gruppiert:

- **Verschachtelte Schleifen:**
 - Äußere Schleife: Über Typen
 - Innere Schleife: Über alle Bauteile
- In der inneren Schleife:
 - Prüfen, ob Bauteil vom aktuellen Typ ist
 - Falls ja: Zähler für Typ erhöhen
 - Falls zusätzlich defekt: Defekt-Zähler erhöhen
- **Wichtig:** Zähler werden für jeden Typ neu auf 0 gesetzt!

Teil 3 – Defekte Bauteile:

- Einfache Filterung: Nur defekte ausgeben

Verschachtelte Schleifen verstehen:

Bei Typ "Schraube" läuft die innere Schleife über **alle** Bauteile und zählt nur die Schrauben: - B001: Typ = Schraube -> gesamt_typ += 1 (gesamt_typ = 1) - B002: Typ = Mutter -> nicht zählen - B003: Typ = Schraube, DEFEKT -> gesamt_typ += 1, defekt_typ += 1 - ...

Verwendete Konzepte:

- Liste von Dictionaries
- Verschachtelte for-Schleifen
- Dictionary-Zugriff
- Mehrfache Bedingungen
- Zähler für verschiedene Kategorien

15 Typische Anfängerfehler – Troubleshooting

Fehler	Ursache	Lösung
IndentationError	Falsche Einrückung in if/while/for	Alle Zeilen im Block müssen gleich eingerückt sein (4 Leerzeichen)
Endlosschleife bei while	Variable in Bedingung wird nicht verändert	Bedingung muss irgendwann False werden (z.B. Zähler verringern)
IndexError: list index out of range	Zugriff auf nicht existierenden Index	Index prüfen, bei for-Schleifen über Länge iterieren
KeyError: 'schlüssel'	Dictionary-Schlüssel existiert nicht	Schreibweise prüfen (Groß-/Kleinschreibung!)
break/continue außerhalb Schleife	break/continue in if ohne Schleife	Nur innerhalb von while/for verwenden
match-case funktioniert nicht	Python-Version < 3.10	Python aktualisieren oder if-elif verwenden
Vergleich mit = statt ==	Zuweisung statt Vergleich	if x == 5: nicht if x = 5:

16 Abschluss & Reflexion

Aufwand: ca. 10 Minuten

Was Sie gelernt haben

In dieser Übung haben Sie: 1. **Programmverzweigungen** mit if, elif und match-case implementiert 2. **Bedingte Wiederholungen** mit while-Schleifen umgesetzt 3. **Iteration** über Kollektionen mit for-Schleifen praktiziert 4. **Schleifensteuerung** mit break und continue gezielt eingesetzt 5. **Komplexe Problemstellungen** durch Kombination aller Konzepte gelöst

Reflexionsfragen

Nehmen Sie sich 5 Minuten Zeit und beantworten Sie für sich:

1. **Welche Aufgabe war am schwierigsten?** Warum?
2. **Wann verwenden Sie while, wann for?** Können Sie die Unterschiede erklären?
3. **Welche Fehler haben Sie gemacht?** Was haben Sie daraus gelernt?
4. **Wo haben break/continue geholfen?** Hätten Sie es auch ohne geschafft?
5. **Wie könnten die komplexen Aufgaben verbessert werden?** Mit Funktionen? Mit anderen Strukturen?

Selbsteinschätzung

Bewerten Sie Ihr Verständnis:

Konzept	* Unsicher	** Geht so	*** Sicher
if-elif-else			
match-case			
while-Schleifen			
for-Schleifen			
break/continue			
Verschachtelte Strukturen			

Empfehlung: Falls Sie bei einem Konzept unsicher sind, wiederholen Sie die entsprechenden Aufgaben oder schauen Sie in die Notebooks 09-10.

Ausblick

Im nächsten Schritt (Notebook 11) lernen Sie: - **Fehlerbehandlung** mit try-except - **Exception-Typen** verstehen - **Robuste Programme** schreiben, die Fehler elegant abfangen

Mit Fehlerbehandlung werden Ihre Programme produktionsreif!

Ende der Übungseinheit

Speichern Sie Ihre Lösungen und vergleichen Sie sie mit den Musterlösungen. Bei Fragen wenden Sie sich an Ihre Übungsleitung oder das Forum.

Viel Erfolg beim weiteren Lernen!